

## Paso 1 — Validar rápidamente la nueva muestra

Antes de derivar nada:

```
SELECT
  COUNT(*) AS row_count,
  approx_count_distinct(party_id) AS party_count,
  approx_count_distinct(acct_id) AS account_count,
  MIN(data_date) AS minimum_data_date,
  MAX(data_date) AS maximum_data_date,
  COUNT(DISTINCT data_date) AS distinct_data_dates
FROM bu_frd.trades_behavior_sample_2024_2025;
```

Esperamos aproximadamente **24 data\_date mensuales**, desde enero de 2024 hasta diciembre de 2025.

También confirma nuevamente la granularidad:

```
WITH duplicates AS (
  SELECT
    party_id,
    acct_id,
    data_date,
    COUNT(*) AS cnt
  FROM bu_frd.trades_behavior_sample_2024_2025
  GROUP BY
    party_id,
    acct_id,
    data_date
  HAVING COUNT(*) > 1
)
SELECT COUNT(*) AS duplicate_key_count
FROM duplicates;
```

Esperamos **0**.

---

## Paso 2 — Crear las transiciones MOP 2024–2025

Ahora reproducimos la lógica que ya validamos, pero sobre los 24 meses.

```
DROP TABLE IF EXISTS bu_frd.trades_mop_transition_2024_2025;
```

```
CREATE TABLE bu_frd.trades_mop_transition_2024_2025
USING PARQUET
AS
```

WITH base AS (

SELECT

party\_id,  
acct\_id,  
data\_date,  
eff\_dte,

acct\_mop\_cde,  
acct\_typ\_cde,  
porf\_typ\_cde,  
kob\_posn\_1\_cde,  
kob\_posn\_2\_cde,

open\_dte,  
clsd\_dte,  
paid\_out\_dte,

curr\_bal\_amt,  
cr\_lmt\_amt,  
high\_cr\_amt,  
past\_due\_amt,

payt\_amt,  
payt\_dte,

LAG(acct\_mop\_cde) OVER (  
PARTITION BY party\_id, acct\_id  
ORDER BY data\_date  
) AS previous\_mop,

LAG(eff\_dte) OVER (  
PARTITION BY party\_id, acct\_id  
ORDER BY data\_date  
) AS previous\_eff\_dte,

LAG(curr\_bal\_amt) OVER (  
PARTITION BY party\_id, acct\_id  
ORDER BY data\_date  
) AS previous\_balance,

LAG(cr\_lmt\_amt) OVER (  
PARTITION BY party\_id, acct\_id  
ORDER BY data\_date  
) AS previous\_credit\_limit,

LAG(past\_due\_amt) OVER (

```

        PARTITION BY party_id, acct_id
        ORDER BY data_date
    ) AS previous_past_due

FROM bu_frd.trades_behavior_sample_2024_2025
),

mapped AS (

SELECT
    *,

    CASE
        WHEN previous_mop = '01' THEN 0
        WHEN previous_mop = '02' THEN 1
        WHEN previous_mop = '03' THEN 2
        WHEN previous_mop = '04' THEN 3
        WHEN previous_mop = '05' THEN 4
        WHEN previous_mop = '06' THEN 5
        ELSE NULL
    END AS previous_delinquency_level,

    CASE
        WHEN acct_mop_cde = '01' THEN 0
        WHEN acct_mop_cde = '02' THEN 1
        WHEN acct_mop_cde = '03' THEN 2
        WHEN acct_mop_cde = '04' THEN 3
        WHEN acct_mop_cde = '05' THEN 4
        WHEN acct_mop_cde = '06' THEN 5
        ELSE NULL
    END AS current_delinquency_level

FROM base
)

SELECT
    *,

    /* Cuenta activa en este snapshot */
    CASE
        WHEN (open_dte IS NULL OR open_dte <= data_date)
            AND (clsd_dte IS NULL OR clsd_dte > data_date)
        THEN 1
        ELSE 0
    END AS active_account_flag,

    /* Nueva actualización efectiva */
    CASE

```

```
    WHEN previous_eff_dte IS NOT NULL
      AND eff_dte <> previous_eff_dte
    THEN 1
    ELSE 0
  END AS effective_update_flag,
```

```
/* Deterioro ordinal 01 -> 06 */
```

```
CASE
  WHEN previous_delinquency_level IS NOT NULL
    AND current_delinquency_level IS NOT NULL
    AND current_delinquency_level >
      previous_delinquency_level
  THEN 1
  ELSE 0
END AS dpd_deterioration_flag,
```

```
/* Mejora ordinal */
```

```
CASE
  WHEN previous_delinquency_level IS NOT NULL
    AND current_delinquency_level IS NOT NULL
    AND current_delinquency_level <
      previous_delinquency_level
  THEN 1
  ELSE 0
END AS dpd_improvement_flag,
```

```
/* Magnitud del cambio */
```

```
CASE
  WHEN previous_delinquency_level IS NOT NULL
    AND current_delinquency_level IS NOT NULL
  THEN current_delinquency_level
    - previous_delinquency_level
  ELSE NULL
END AS dpd_level_change,
```

```
/* Entrada a adverse activo */
```

```
CASE
  WHEN acct_mop_cde IN ('09','9B','8A','8B')
    AND COALESCE(previous_mop,"") NOT IN (
      '09','9B','8A','8B'
    )
  THEN 1
  ELSE 0
END AS entered_active_adverse_flag,
```

```
/* Adverse pagado/resuelto */
```

```
CASE
  WHEN acct_mop_cde IN ('9P','8P')
```

```

        AND COALESCE(previous_mop,") <> acct_mop_cde
    THEN 1
    ELSE 0
END AS adverse_resolved_flag,

```

```

CASE
    WHEN acct_mop_cde = 'UR'
        AND COALESCE(previous_mop,") <> 'UR'
    THEN 1
    ELSE 0
END AS became_unrated_flag,

```

```

CASE
    WHEN previous_mop = 'UR'
        AND acct_mop_cde <> 'UR'
    THEN 1
    ELSE 0
END AS left_unrated_flag

```

FROM mapped;

Hay una ventaja importante: las transiciones de 2025 ahora tendrán acceso a diciembre/meses anteriores de 2024. Esto hace mucho más robusto el análisis longitudinal.

---

## Paso 3 — Validar que 2025 siga comportándose igual

Antes de agregar a persona, quiero confirmar que al ampliar a 24 meses no rompimos nada:

```

SELECT
    SUM(
        CASE
            WHEN dpd_deterioration_flag = 1
                AND effective_update_flag = 1
            THEN 1 ELSE 0
        END
    ) AS deterioration_2025,

    SUM(
        CASE
            WHEN dpd_improvement_flag = 1
                AND effective_update_flag = 1
            THEN 1 ELSE 0
        END
    ) AS improvement_2025,

    SUM(

```

```

CASE
  WHEN entered_active_adverse_flag = 1
    AND effective_update_flag = 1
  THEN 1 ELSE 0
END
) AS active_adverse_2025

FROM bu_frd.trades_mop_transition_2024_2025

WHERE YEAR(data_date) = 2025;

```

No necesariamente será idéntico al análisis anterior, porque ahora enero de 2025 puede compararse correctamente con diciembre de 2024. De hecho, **esperaría que aparezcan algunos eventos adicionales en enero de 2025**.

Eso sería una mejora, no un problema.

---

## Paso 4 — Crear la tabla **party × month**

Ahora sí:

```

DROP TABLE IF EXISTS bu_frd.party_trade_behavior_monthly_2024_2025;

CREATE TABLE bu_frd.party_trade_behavior_monthly_2024_2025
USING PARQUET
AS

SELECT
  party_id,
  data_date,

  /* Portfolio */
  COUNT(DISTINCT acct_id)
    AS total_accounts,

  SUM(active_account_flag)
    AS active_accounts,

  SUM(
    CASE
      WHEN clsd_dte IS NOT NULL
        AND clsd_dte <= data_date
      THEN 1 ELSE 0
    )

```

```

) AS closed_accounts,

/* Exposure */
SUM(
  CASE
    WHEN active_account_flag = 1
    THEN COALESCE(curr_bal_amt,0)
    ELSE 0
  END
) AS total_active_balance,

SUM(
  CASE
    WHEN active_account_flag = 1
    THEN COALESCE(past_due_amt,0)
    ELSE 0
  END
) AS total_past_due_amount,

/* Delinquency state */
SUM(
  CASE
    WHEN active_account_flag = 1
    AND acct_mop_cde IN ('02','03','04','05','06')
    THEN 1 ELSE 0
  END
) AS delinquent_accounts,

SUM(
  CASE
    WHEN active_account_flag = 1
    AND acct_mop_cde IN ('03','04','05','06')
    THEN 1 ELSE 0
  END
) AS dpd_60plus_accounts,

SUM(
  CASE
    WHEN active_account_flag = 1
    AND acct_mop_cde IN ('04','05','06')
    THEN 1 ELSE 0
  END
) AS dpd_90plus_accounts,

/* Behavioral events */
SUM(
  CASE
    WHEN dpd_deterioration_flag = 1

```

```

        AND effective_update_flag = 1
        THEN 1 ELSE 0
    END
) AS deteriorated_accounts,

SUM(
    CASE
        WHEN dpd_improvement_flag = 1
            AND effective_update_flag = 1
        THEN 1 ELSE 0
    END
) AS improved_accounts,

SUM(
    CASE
        WHEN entered_active_adverse_flag = 1
            AND effective_update_flag = 1
        THEN 1 ELSE 0
    END
) AS entered_active_adverse_accounts,

SUM(
    CASE
        WHEN adverse_resolved_flag = 1
            AND effective_update_flag = 1
        THEN 1 ELSE 0
    END
) AS adverse_resolved_accounts,

SUM(
    CASE
        WHEN became_unrated_flag = 1
            AND effective_update_flag = 1
        THEN 1 ELSE 0
    END
) AS became_unrated_accounts,

/* Severity */
MAX(
    CASE
        WHEN dpd_deterioration_flag = 1
            AND effective_update_flag = 1
        THEN dpd_level_change
    END
) AS max_dpd_jump,

SUM(
    CASE

```

```

        WHEN dpd_deterioration_flag = 1
        AND effective_update_flag = 1
        THEN dpd_level_change
        ELSE 0
    END
) AS total_dpd_deterioration_points,

/* Freshness */
COUNT(
    DISTINCT CASE
        WHEN effective_update_flag = 1
        THEN acct_id
    END
) AS effectively_updated_accounts,

/* Revolving / line-of-credit exposure only */
SUM(
    CASE
        WHEN active_account_flag = 1
        AND porf_typ_cde IN ('R','C')
        THEN COALESCE(curr_bal_amt,0)
        ELSE 0
    END
) AS revolving_balance,

SUM(
    CASE
        WHEN active_account_flag = 1
        AND porf_typ_cde IN ('R','C')
        THEN COALESCE(cr_lmt_amt,0)
        ELSE 0
    END
) AS revolving_credit_limit

```

FROM bu\_frd.trades\_mop\_transition\_2024\_2025

GROUP BY  
 party\_id,  
 data\_date;

Aquí hice una corrección respecto a la versión anterior: **no mezclamos todas las cuentas para utilización**. Guardamos específicamente **revolving\_balance** y **revolving\_credit\_limit**, porque **cr\_lmt\_amt** tiene mucho más sentido para **R/C** que para installment o mortgage.

---

## Paso 5 — Agregar ratios

```
DROP TABLE IF EXISTS bu_frd.party_trade_behavior_monthly_2024_2025_v2;
```

```
CREATE TABLE bu_frd.party_trade_behavior_monthly_2024_2025_v2  
USING PARQUET  
AS
```

```
SELECT
```

```
*,
```

```
CASE
```

```
  WHEN active_accounts > 0
```

```
  THEN 1.0 * delinquent_accounts / active_accounts
```

```
END AS pct_accounts_delinquent,
```

```
CASE
```

```
  WHEN active_accounts > 0
```

```
  THEN 1.0 * dpd_60plus_accounts / active_accounts
```

```
END AS pct_accounts_60plus,
```

```
CASE
```

```
  WHEN active_accounts > 0
```

```
  THEN 1.0 * dpd_90plus_accounts / active_accounts
```

```
END AS pct_accounts_90plus,
```

```
CASE
```

```
  WHEN active_accounts > 0
```

```
  THEN 1.0 * deteriorated_accounts / active_accounts
```

```
END AS pct_accounts_deteriorated,
```

```
CASE
```

```
  WHEN active_accounts > 0
```

```
  THEN 1.0 * entered_active_adverse_accounts  
        / active_accounts
```

```
END AS pct_accounts_entered_adverse,
```

```
CASE
```

```
  WHEN active_accounts > 0
```

```
  THEN 1.0 * effectively_updated_accounts  
        / active_accounts
```

```
END AS pct_accounts_effectively_updated,
```

```
CASE
```

```
  WHEN revolving_credit_limit > 0
```

```
  THEN 1.0 * revolving_balance  
        / revolving_credit_limit
```

```
END AS revolving_utilization
```

```
FROM bu_frd.party_trade_behavior_monthly_2024_2025;
```

---

## Paso 6 — Validaciones antes de construir ventanas

Primero:

```
WITH duplicates AS (  
  SELECT  
    party_id,  
    data_date,  
    COUNT(*) AS cnt  
  FROM bu_frd.party_trade_behavior_monthly_2024_2025_v2  
  GROUP BY party_id, data_date  
  HAVING COUNT(*) > 1  
)  
SELECT COUNT(*) AS duplicate_party_month_count  
FROM duplicates;
```

Debe dar 0.

Luego:

```
SELECT  
  COUNT(*) AS row_count,  
  COUNT(DISTINCT party_id) AS party_count,  
  MIN(data_date) AS minimum_data_date,  
  MAX(data_date) AS maximum_data_date,  
  COUNT(DISTINCT data_date) AS distinct_months  
FROM bu_frd.party_trade_behavior_monthly_2024_2025_v2;
```

Y finalmente quiero ver cobertura mensual:

```
SELECT  
  data_date,  
  COUNT(*) AS party_count,  
  ROUND(AVG(active_accounts),2) AS avg_active_accounts,  
  SUM(deteriorated_accounts) AS deteriorated_accounts,  
  SUM(entered_active_adverse_accounts) AS adverse_accounts  
FROM bu_frd.party_trade_behavior_monthly_2024_2025_v2  
GROUP BY data_date  
ORDER BY data_date;
```

Esta última validación es especialmente importante. Queremos comprobar que no haya un mes con una caída o incremento extraño debido al pipeline.

## Después de estos resultados

El siguiente paso será probablemente el más importante hasta ahora: crear para cada `party_id + observation_date` variables como:

deteriorations\_3m  
deteriorations\_6m  
deteriorations\_12m

adverse\_events\_3m  
past\_due\_avg\_3m  
past\_due\_avg\_12m

utilization\_avg\_3m  
utilization\_avg\_12m

recent\_3m\_vs\_prior\_9m

Pero primero pásame los resultados del **Paso 3 y de las validaciones del Paso 6**. Si esos números se ven estables, construiremos las ventanas rolling sin volver a tocar la tabla gigantesca `trades`.

## Paso 7 — Crear rolling windows 3m, 6m y 12m

Aquí hay una consideración importante: **no usaría simplemente `ROWS BETWEEN 2 PRECEDING`**, porque algunas personas pueden faltar en algún mes.

En su lugar vamos a crear un índice numérico del mes y usar `RANGE`. De esa manera, 3 meses significa realmente tres meses calendario.

```
DROP TABLE IF EXISTS bu_frd.party_trade_behavior_rolling_2025;
```

```
CREATE TABLE bu_frd.party_trade_behavior_rolling_2025  
USING PARQUET  
AS
```

```
WITH base AS (
```

```
    SELECT
```

```

*,

YEAR(data_date) * 12
+ MONTH(data_date) AS month_index

FROM bu_frd.party_trade_behavior_monthly_2024_2025_v2
),

rolling AS (

SELECT
*,

/* =====
DATA COVERAGE
===== */

COUNT(*) OVER (
PARTITION BY party_id
ORDER BY month_index
RANGE BETWEEN 2 PRECEDING AND CURRENT ROW
) AS observed_months_3m,

COUNT(*) OVER (
PARTITION BY party_id
ORDER BY month_index
RANGE BETWEEN 5 PRECEDING AND CURRENT ROW
) AS observed_months_6m,

COUNT(*) OVER (
PARTITION BY party_id
ORDER BY month_index
RANGE BETWEEN 11 PRECEDING AND CURRENT ROW
) AS observed_months_12m,

/* =====
DETERIORATION EVENTS
===== */

SUM(deteriorated_accounts) OVER (
PARTITION BY party_id
ORDER BY month_index
RANGE BETWEEN 2 PRECEDING AND CURRENT ROW
) AS deteriorations_3m,

SUM(deteriorated_accounts) OVER (
PARTITION BY party_id

```

```
ORDER BY month_index  
RANGE BETWEEN 5 PRECEDING AND CURRENT ROW  
) AS deteriorations_6m,
```

```
SUM(deteriorated_accounts) OVER (  
PARTITION BY party_id  
ORDER BY month_index  
RANGE BETWEEN 11 PRECEDING AND CURRENT ROW  
) AS deteriorations_12m,
```

```
/* =====  
ACTIVE ADVERSE EVENTS  
===== */
```

```
SUM(entered_active_adverse_accounts) OVER (  
PARTITION BY party_id  
ORDER BY month_index  
RANGE BETWEEN 2 PRECEDING AND CURRENT ROW  
) AS active_adverse_3m,
```

```
SUM(entered_active_adverse_accounts) OVER (  
PARTITION BY party_id  
ORDER BY month_index  
RANGE BETWEEN 5 PRECEDING AND CURRENT ROW  
) AS active_adverse_6m,
```

```
SUM(entered_active_adverse_accounts) OVER (  
PARTITION BY party_id  
ORDER BY month_index  
RANGE BETWEEN 11 PRECEDING AND CURRENT ROW  
) AS active_adverse_12m,
```

```
/* =====  
PAST DUE  
state variable -> average  
===== */
```

```
AVG(total_past_due_amount) OVER (  
PARTITION BY party_id  
ORDER BY month_index  
RANGE BETWEEN 2 PRECEDING AND CURRENT ROW  
) AS avg_past_due_3m,
```

```
AVG(total_past_due_amount) OVER (  
PARTITION BY party_id  
ORDER BY month_index
```

RANGE BETWEEN 11 PRECEDING AND CURRENT ROW  
) AS avg\_past\_due\_12m,

/\* =====  
BALANCE  
===== \*/

AVG(total\_active\_balance) OVER (  
PARTITION BY party\_id  
ORDER BY month\_index  
RANGE BETWEEN 2 PRECEDING AND CURRENT ROW  
) AS avg\_balance\_3m,

AVG(total\_active\_balance) OVER (  
PARTITION BY party\_id  
ORDER BY month\_index  
RANGE BETWEEN 11 PRECEDING AND CURRENT ROW  
) AS avg\_balance\_12m,

/\* =====  
DELINQUENCY SHARE  
===== \*/

AVG(pct\_accounts\_delinquent) OVER (  
PARTITION BY party\_id  
ORDER BY month\_index  
RANGE BETWEEN 2 PRECEDING AND CURRENT ROW  
) AS avg\_pct\_delinquent\_3m,

AVG(pct\_accounts\_delinquent) OVER (  
PARTITION BY party\_id  
ORDER BY month\_index  
RANGE BETWEEN 11 PRECEDING AND CURRENT ROW  
) AS avg\_pct\_delinquent\_12m,

/\* =====  
ACTIVE ACCOUNT EXPOSURE  
===== \*/

AVG(active\_accounts) OVER (  
PARTITION BY party\_id  
ORDER BY month\_index  
RANGE BETWEEN 2 PRECEDING AND CURRENT ROW  
) AS avg\_active\_accounts\_3m,

```
AVG(active_accounts) OVER (  
  PARTITION BY party_id  
  ORDER BY month_index  
  RANGE BETWEEN 11 PRECEDING AND CURRENT ROW  
) AS avg_active_accounts_12m,
```

```
/* =====  
REVOLVING UTILIZATION  
===== */
```

```
AVG(revolving_utilization) OVER (  
  PARTITION BY party_id  
  ORDER BY month_index  
  RANGE BETWEEN 2 PRECEDING AND CURRENT ROW  
) AS avg_revolving_utilization_3m,
```

```
AVG(revolving_utilization) OVER (  
  PARTITION BY party_id  
  ORDER BY month_index  
  RANGE BETWEEN 11 PRECEDING AND CURRENT ROW  
) AS avg_revolving_utilization_12m,
```

```
/* =====  
MAXIMUM DPD JUMP  
===== */
```

```
MAX(max_dpd_jump) OVER (  
  PARTITION BY party_id  
  ORDER BY month_index  
  RANGE BETWEEN 2 PRECEDING AND CURRENT ROW  
) AS max_dpd_jump_3m,
```

```
MAX(max_dpd_jump) OVER (  
  PARTITION BY party_id  
  ORDER BY month_index  
  RANGE BETWEEN 11 PRECEDING AND CURRENT ROW  
) AS max_dpd_jump_12m
```

```
FROM base  
)
```

```
SELECT *  
FROM rolling
```

```
/* 2024 is history; observations start in 2025 */  
WHERE data_date >= DATE('2025-01-31');
```

---

## Paso 8 — No usar observaciones sin suficiente historia

Esto es muy importante.

Para modelar, un party con:

observed\_months\_12m = 3

no es comparable con alguien con:

observed\_months\_12m = 12

Por eso vamos a crear un indicador:

```
DROP TABLE IF EXISTS bu_frd.party_trade_features_2025_stage1;
```

```
CREATE TABLE bu_frd.party_trade_features_2025_stage1
USING PARQUET
AS
```

```
SELECT
```

```
  *,
```

```
  CASE
```

```
    WHEN observed_months_3m = 3
      AND observed_months_6m = 6
      AND observed_months_12m = 12
```

```
    THEN 1
```

```
    ELSE 0
```

```
  END AS complete_history_flag
```

```
FROM bu_frd.party_trade_behavior_rolling_2025;
```

No eliminaría todavía las observaciones incompletas. Las conservaría para analizar cobertura.

---

## Paso 9 — Cuánta población tiene historia completa

Ejecuta:

```
SELECT
```

```

complete_history_flag,

COUNT(*) AS party_month_count,

COUNT(DISTINCT party_id) AS party_count,

ROUND(
  100.0 * COUNT(*)
  / SUM(COUNT(*)) OVER (),
  2
) AS pct_party_months

FROM bu_frd.party_trade_features_2025_stage1

GROUP BY complete_history_flag;

```

Quiero saber qué porcentaje de nuestros **party-month** 2025 realmente dispone de los 12 meses anteriores.

---

## Paso 10 — Ahora viene la feature más importante: cambio reciente vs baseline

Hasta ahora hicimos:

3m  
6m  
12m

pero esas ventanas se solapan.

Para detectar **aceleración del comportamiento**, prefiero comparar:

RECENT:  
últimos 3 meses

vs.

BASELINE:  
los 9 meses anteriores

Es decir:

|----- 12 meses -----|

|----- prior 9m -----|--- recent 3m ---|

Vamos a calcular directamente el **prior 9m**.

```
DROP TABLE IF EXISTS bu_frd.party_trade_features_2025_stage2;
```

```
CREATE TABLE bu_frd.party_trade_features_2025_stage2
USING PARQUET
AS
```

```
WITH base AS (
```

```
    SELECT
```

```
        *,
```

```
        YEAR(data_date) * 12
```

```
        + MONTH(data_date) AS month_index
```

```
    FROM bu_frd.party_trade_behavior_monthly_2024_2025_v2
```

```
),
```

```
features AS (
```

```
    SELECT
```

```
        *,
```

```
        /* ----- Recent 3 months ----- */
```

```
        SUM(deteriorated_accounts) OVER (
```

```
            PARTITION BY party_id
```

```
            ORDER BY month_index
```

```
            RANGE BETWEEN 2 PRECEDING AND CURRENT ROW
```

```
        ) AS deteriorations_recent_3m,
```

```
        SUM(entered_active_adverse_accounts) OVER (
```

```
            PARTITION BY party_id
```

```
            ORDER BY month_index
```

```
            RANGE BETWEEN 2 PRECEDING AND CURRENT ROW
```

```
        ) AS adverse_recent_3m,
```

```
        AVG(total_past_due_amount) OVER (
```

```
            PARTITION BY party_id
```

```
            ORDER BY month_index
```

```
            RANGE BETWEEN 2 PRECEDING AND CURRENT ROW
```

```
        ) AS avg_past_due_recent_3m,
```

```
        AVG(total_active_balance) OVER (
```

```
            PARTITION BY party_id
```

```
ORDER BY month_index  
RANGE BETWEEN 2 PRECEDING AND CURRENT ROW  
) AS avg_balance_recent_3m,
```

```
AVG(pct_accounts_delinquent) OVER (  
PARTITION BY party_id  
ORDER BY month_index  
RANGE BETWEEN 2 PRECEDING AND CURRENT ROW  
) AS avg_pct_delinquent_recent_3m,
```

```
/* ----- Previous 9 months ----- */
```

```
SUM(deteriorated_accounts) OVER (  
PARTITION BY party_id  
ORDER BY month_index  
RANGE BETWEEN 11 PRECEDING AND 3 PRECEDING  
) AS deteriorations_prior_9m,
```

```
SUM(entered_active_adverse_accounts) OVER (  
PARTITION BY party_id  
ORDER BY month_index  
RANGE BETWEEN 11 PRECEDING AND 3 PRECEDING  
) AS adverse_prior_9m,
```

```
AVG(total_past_due_amount) OVER (  
PARTITION BY party_id  
ORDER BY month_index  
RANGE BETWEEN 11 PRECEDING AND 3 PRECEDING  
) AS avg_past_due_prior_9m,
```

```
AVG(total_active_balance) OVER (  
PARTITION BY party_id  
ORDER BY month_index  
RANGE BETWEEN 11 PRECEDING AND 3 PRECEDING  
) AS avg_balance_prior_9m,
```

```
AVG(pct_accounts_delinquent) OVER (  
PARTITION BY party_id  
ORDER BY month_index  
RANGE BETWEEN 11 PRECEDING AND 3 PRECEDING  
) AS avg_pct_delinquent_prior_9m,
```

```
COUNT(*) OVER (  
PARTITION BY party_id  
ORDER BY month_index  
RANGE BETWEEN 11 PRECEDING AND 3 PRECEDING  
) AS observed_months_prior_9m
```

```

FROM base
)

SELECT
*,

/* Compare monthly event intensity.
   Divide by window length so 3m and 9m are comparable. */

(
  COALESCE(deteriorations_recent_3m,0) / 3.0
  -
  COALESCE(deteriorations_prior_9m,0) / 9.0
) AS deterioration_acceleration,

(
  COALESCE(adverse_recent_3m,0) / 3.0
  -
  COALESCE(adverse_prior_9m,0) / 9.0
) AS adverse_acceleration,

(
  avg_past_due_recent_3m
  -
  avg_past_due_prior_9m
) AS past_due_change_vs_baseline,

(
  avg_balance_recent_3m
  -
  avg_balance_prior_9m
) AS balance_change_vs_baseline,

(
  avg_pct_delinquent_recent_3m
  -
  avg_pct_delinquent_prior_9m
) AS delinquency_share_change_vs_baseline

```

FROM features

WHERE data\_date >= DATE('2025-01-31');

Esta tabla empieza a representar mucho mejor el concepto del índice.

Por ejemplo:

Party A

Prior 9m:  
0 deterioros

Recent 3m:  
3 deterioros

deterioration\_acceleration = +1.0 por mes

mientras:

Party B

Prior 9m:  
9 deterioros

Recent 3m:  
3 deterioros

recent rate = 1 / month  
baseline rate = 1 / month

deterioration\_acceleration = 0

Los dos tuvieron tres deterioros recientes, pero **solo A cambió su comportamiento**.

Eso es exactamente la clase de señal que queremos.

---

## Hay una señal temporal que debemos investigar

Tu cobertura mensual muestra que los deterioros aumentan notablemente a principios de 2025:

Dic-2024 ~2,102  
Jan-2025 ~2,130  
Feb-2025 ~3,079  
Mar-2025 ~3,290  
Apr-2025 ~3,287  
May-2025 ~2,922  
Jun-2025 ~2,296

El número de parties cambia muy poco, así que **no parece explicado simplemente por aumento de población**.

No concluiría que existe más vulnerabilidad en Q1/Q2 2025. Puede ser:

- estacionalidad,
- cambios de reporting,
- composición de productos,
- una transición MOP específica,
- cambio en proveedor/lógica.

Antes de entrenar un modelo tendremos que medir **drift por calendario**. Pero no bloquea la creación de las features rolling.

## Qué ejecutaría ahora

Primero crea `party_trade_behavior_rolling_2025` y `party_trade_features_2025_stage1`. Luego pásame únicamente el resultado de:

```
complete_history_flag = 0/1  
party_month_count  
party_count  
pct_party_months
```

Con eso comprobamos cuánta muestra tiene baseline completo. Si la cobertura es buena, seguimos con `stage2` y hacemos el primer EDA de **deterioration acceleration / past-due change / balance change** antes de cruzarlo con fraud alerts.